

Complete Textbook Notes on Interview Q and A

Day 9 — Trees, Backend Query Abstraction, API Reliability & System Design

Professional textbook edition containing the complete preceding Interview Q&A; conversation verbatim, followed by clearly separated supplementary learning aids.

Edition	Day 9 Interview Preparation
Primary focus	DSA + Backend + Reliability + System Design
Source integrity	Verbatim source preserved in sequence

Table of Contents

The source conversation is retained in its original sequence. Chapter and question headings below are navigational structure added by the textbook formatting layer.

- PART I — TREES, DFS & BFS
- PART II — DSA PROBLEM 1: BINARY TREE PREORDER
- PART III — DSA PROBLEM 2: BINARY TREE LEVEL ORDER
- PART IV — DSA PROBLEM 3: MAXIMUM DEPTH
- PART V — BACKEND QUERY ABSTRACTION
- PART VI — API RELIABILITY
- PART VII — SYSTEM DESIGN
- PART VIII — PROJECT INTERVIEW QUESTIONS
- PART IX — GOOGLE-STYLE FOLLOW-UP QUESTIONS
- PART X — RAPID-FIRE QUESTIONS
- PART XI — SCENARIO-BASED INTERVIEW QUESTIONS
- PART XII — DEEP PROJECT QUESTION
- PART XIII — MOST IMPORTANT INTERVIEW CONNECTIONS
- PART XIV — DAY 9 MOCK INTERVIEW
- PART XV — THE 10 QUESTIONS YOU MUST MASTER

Supplementary Learning Aids

Chapter Summaries

Revision Checklist

Glossary

Index

VERBATIM CONVERSATION — START

Everything from this point through the matching END marker is the preceding conversation content, with formatting adapted for textbook presentation. No source wording is intentionally summarized or replaced.

give me the detailed interview Q and A for Day9 tasks.

Day 9 — Detailed Interview Questions & Answers

Day 9 is centered on four connected interview areas:

1. Trees + DFS/BFS
2. Backend Query Abstraction
3. API Reliability & Failure Handling
4. System Design + Project 1 Architecture

INTERVIEW PREPARATION PRINCIPLE

The goal is not just to memorize answers. For Google-level interviews, you should be able to explain the reasoning, choose the appropriate pattern, discuss complexity, identify failure modes, and connect the concept to your project.

PART I — TREES, DFS & BFS

Verbatim conversation content — formatted for textbook presentation

PART I — TREES, DFS & BFS

Q1. What is a tree?

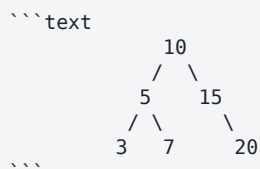
Answer

A tree is a hierarchical data structure consisting of nodes connected by edges.

A tree has:

- One root
- Zero or more child nodes for each node
- No cycles
- Exactly one path between any two connected nodes

Example:



Here:

- 10 → root
- 5 and 15 → children of 10
- 3, 7, 20 → leaf nodes
- 5 → parent of 3 and 7
- 3 → child of 5

Interview point

A tree is essentially a hierarchical graph with connectivity and no cycles.

Q2. What is a binary tree?

Answer

A binary tree is a tree in which each node can have at most two children.

Those children are conventionally called:

```
``text
left child
right child
````
```

Example:

```

```text
      A
     / \
    B   C
   / \
  D   E
```

```

Each node contains:

```

```text
value
left
right
```

```

A conceptual Python representation is:

```

```python
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

```

## Important distinction

A binary tree is not necessarily a Binary Search Tree.

A binary tree only requires:

```

```text
at most two children
```

```

A BST additionally requires an ordering property.

# Q3. What is the difference between depth and height?

## Answer

These terms are related but measure different things.

## Depth

The depth of a node is the number of edges from the root to that node.

```

```text
      A                depth = 0
     / \
    B   C             depth = 1
   /
  D                   depth = 2
```

```

Therefore:

```

```text
depth(A) = 0
depth(B) = 1
depth(D) = 2
```

```

## Height

The height of a node is the number of edges on the longest path from that node down to a leaf.

For example:

```
``text
 A
 /
B
/
D
````
```

The height of A is 2.

Interview warning

Different textbooks sometimes define height using number of nodes rather than edges.

Always clarify the convention if the interviewer has not specified it.

Q4. What is DFS in a tree?

Answer

Depth-First Search (DFS) explores one branch as deeply as possible before moving to another branch.

Example:

```
``text
  A
 / \
B   C
/ \
D   E
````
```

One DFS traversal can produce:

```
``text
A B D E C
````
```

DFS naturally maps to:

- Recursion
- Explicit stack

The recursive structure is:

```
``text
process current node
  process left subtree
  process right subtree
````
```

---

# Q5. Why does DFS naturally use recursion?

## Answer

A tree is recursively defined:

*A tree consists of a node and zero or more subtrees.*

For a binary tree:

```
```text
Tree
├── root
├── left subtree
└── right subtree
```
```

Therefore the recursive solution naturally becomes:

```
```python
def dfs(node):
    if node is None:
        return

    dfs(node.left)
    dfs(node.right)
```
```

Every recursive call represents moving one level deeper into the tree.

The call stack keeps track of the nodes whose remaining subtrees still need to be processed.

---

# Q6. What is the space complexity of recursive DFS?

## Answer

If the tree has height  $h$ , recursive DFS uses:

```
```text
O(h)
```
```

call-stack space.

For a balanced tree:

```
```text
h = O(log n)
```
```

So auxiliary space is:

```
```text
O(log n)
```
```

For a completely skewed tree:

```
```text
h = O(n)
```
```

So auxiliary space becomes:

```
```text
O(n)
```
```

## Important interview distinction

Traversal itself takes:

```
```text
O(n)
```
```

time because every node is visited.

But auxiliary recursion space is:

```
```text
O(h)
```
```

---

# Q7. What are preorder, inorder and postorder traversal?

## Answer

They describe the position of the root relative to its subtrees.

## Preorder

```
```text
Root → Left → Right
```
```

## Inorder

```
```text
Left → Root → Right
```
```

## Postorder

```
```text
Left → Right → Root
```
```

For:

```
```text
      A
     /\
    B  C
   /\ 
  D  E
```
```

## Preorder

```
```text
A B D E C
```
```

## Inorder

```
```text
D B E A C
```
```

## Postorder

```
```text
D E B C A
```
```

---

# Q8. When would you choose preorder traversal?

## Answer

Preorder is useful when the parent should be processed before its children.

Conceptually:

```
```text
process parent
then process descendants
```
```

This can be useful when:

- Serializing a tree
- Copying a tree structure
- Processing hierarchical configuration
- Generating representations where parent information comes first

The important recognition pattern is:

*Root first → preorder.*

---

# Q9. When would you choose postorder traversal?

## Answer

Postorder processes children before the parent:

```
```text
Left
Right
Root
```
```

This is useful when the parent depends on information computed from its children.

For example:

```
```text
calculate subtree information
      ↓
calculate parent
```
```

Maximum depth is naturally expressed recursively using this idea:

```
```python
depth = 1 + max(left_depth, right_depth)
```
```

The children must first provide their results.

---

# Q10. What is BFS?

## Answer

Breadth-First Search explores a tree level by level.

Example:

```
```text
      A
     / \
    B   C
   / \ / \
  D  E F
```
```

BFS produces:

```
```text
A
B C
D E F
```
```

or:

```
```text
A B C D E F
```
```

BFS uses a queue.

The basic process is:

```
```text
enqueue root

while queue is not empty:
    remove front node
    process it
    enqueue its children
```
```

---

# Q11. Why does BFS use a queue?

## Answer

A queue follows:

```
```text
FIFO
First In → First Out
```
```

When we process a node, its children are placed at the back of the queue.

Therefore:

```
```text
Current level
  ↓
Children added behind existing nodes
  ↓
Current level finishes
  ↓
Next level begins
```
```

This naturally produces level-order traversal.

---

# Q12. When should you choose BFS instead of DFS?

### Answer

Look at the structure of the problem.

#### Use DFS when:

- The problem involves a subtree
- You need to explore deeply
- The solution is naturally recursive
- You need information from child subtrees
- You need preorder/inorder/postorder

#### Use BFS when:

- The problem explicitly talks about levels
- You need level-order traversal
- You need the closest/shortest number of edges in an unweighted graph/tree
- The problem asks for nodes at distance k

### Recognition shortcut

```
```text
"level by level"
  ↓
  BFS

"subtree"
"deep"
"recursive"
  ↓
  DFS
```
```

# Q13. What is the relationship between trees and graphs?

### Answer

A tree can be viewed as a special type of graph.

A tree is:

- Connected
- Acyclic

For  $n$  nodes, a tree contains:

```
```text
n - 1
```
```

edges.

The conceptual progression is:

```
```text
Trees
  ↓
DFS / BFS
  ↓
Graphs
  ↓
Topological Sort
  ↓
Union-Find
```
```

This is why learning tree traversal first is useful before general graph algorithms.

---

# Q14. How do you recognize that a problem is a DFS problem?

### Answer

Look for language such as:

- Traverse a tree
- Explore every subtree
- Calculate something for every node
- Find maximum/minimum subtree property
- Calculate depth
- Recursively combine left and right results
- Process descendants before parent

For example:

*Find the maximum depth of a binary tree.*

Immediately think:

```
```text
depth(left)
depth(right)
combine results
```
```

Therefore:

```
```text
DFS / recursion
```
```

is a natural pattern.

---

# Q15. How do you recognize a BFS problem?

## Answer

Look for:

- Level order
- Level-by-level
- Minimum number of edges
- Nearest node
- Nodes at distance k
- Shortest path in an unweighted graph

For example:

*Return the nodes of a binary tree level by level.*

The recognition is:

```
```text
level by level
  ↓
  BFS
  ↓
  Queue
```
```

# **PART II — DSA PROBLEM 1: BINARY TREE PREORDER**

Verbatim conversation content — formatted for textbook presentation

## PART II — DSA PROBLEM 1: BINARY TREE PREORDER

# Q16. How would you solve Binary Tree Preorder Traversal?

### Problem

Given:

```
```text
    1
   / \
  2   3
```
```

Return:

```
```text
[1, 2, 3]
```
```

### Pattern

```
```text
Tree DFS
Preorder
```
```

### Recognition

The required order is:

```
```text
Root → Left → Right
```
```

### Recursive solution

```
```python
def preorder(root):
    result = []

    def dfs(node):
        if node is None:
            return

        result.append(node.value)
        dfs(node.left)
        dfs(node.right)

    dfs(root)
    return result
```
```

### Complexity

Every node is visited exactly once:

```
```text
Time: O(n)
```
```

Recursive call stack:

```
```text
Space: O(h)
```
```

where h is tree height.

---

# Q17. How would you implement preorder traversal iteratively?

### Answer

Use an explicit stack.

```
```python
def preorder(root):
    if root is None:
        return []

    result = []
    stack = [root]

    while stack:
        node = stack.pop()
        result.append(node.value)

        if node.right:
            stack.append(node.right)

        if node.left:
            stack.append(node.left)

    return result
```
```

### Why push right before left?

The stack is LIFO.

We want:

```
```text
left before right
```
```

Therefore:

```
```text
push right
push left
```
```

so that:

```
```text
left
```
```

is popped first.

---

# Q18. What is the invariant in iterative preorder traversal?

### Answer

The stack contains nodes that still need to be processed, and the next node popped is the next node in preorder.

Because:

```
```text
Root → Left → Right
```
```

we push:

```
```text
Right
Left
```
```

so Left is processed first.

This is an excellent interview-level explanation because it explains why the algorithm works, not just what it does.

---

## **PART III — DSA PROBLEM 2: BINARY TREE LEVEL ORDER**

Verbatim conversation content — formatted for textbook presentation

## PART III — DSA PROBLEM 2: BINARY TREE LEVEL ORDER

# Q19. How would you solve Binary Tree Level Order Traversal?

Given:

```
```text
      3
     / \
    9  20
   /  \
  15   7
```
```

Return:

```
```text
[
  [3],
  [9, 20],
  [15, 7]
]
```

Pattern

```
```text
Tree
+
BFS
+
Queue
```
```

Algorithm

1. Put root into queue.
2. Determine current queue size.
3. Process exactly that many nodes.
4. Add their children to queue.
5. Those children form the next level.
6. Repeat.

Python

```
```python
from collections import deque

def level_order(root):
 if root is None:
 return []

 result = []
 queue = deque([root])

 while queue:
 level = []

 for _ in range(len(queue)):
 node = queue.popleft()
 level.append(node.value)

 if node.left:
 queue.append(node.left)

 if node.right:
 queue.append(node.right)

 result.append(level)

 return result
```
```

Q20. Why do we use `len(queue)` in level-order traversal?

Answer

This is one of the most important details.

Suppose:

```
```text
queue = [9, 20]
```
```

At that moment, these two nodes belong to the same level.

We capture:

```
```python
level_size = len(queue)
```
```

Then process exactly:

```
```text
level_size
```
```

nodes.

While processing them, we may add:

```
```text
15
7
```
```

to the queue.

Those new nodes belong to the next level, not the current level.

Therefore:

```
```python
for _ in range(level_size):
```
```

creates a boundary between levels.

Q21. What is the complexity of level-order traversal?

Answer

Every node enters and leaves the queue once.

Therefore:

```
```text
Time = $O(n)$
```
```

The maximum queue size depends on the maximum width of the tree.

So auxiliary space is:

```
```text
 $O(w)$
```
```

where w is maximum tree width.

In the worst case:

```
```text
 $O(n)$
```
```

PART IV — DSA PROBLEM 3: MAXIMUM DEPTH

Verbatim conversation content — formatted for textbook presentation

PART IV — DSA PROBLEM 3: MAXIMUM DEPTH

Q22. How would you solve Maximum Depth of Binary Tree?

Given:

```

```text
 3
 / \
 9 20
 / \ \
 15 7
```

```

The maximum depth is:

```

```text
3
```

```

Pattern

```

```text
Tree DFS
+
Recursion
```

```

Recurrence

For a non-empty node:

```

```text
depth(node)
=
1 + max(
 depth(left),
 depth(right)
)
```

```

Base case:

```

```text
depth(None) = 0
```

```

Python

```

```python
def max_depth(root):
 if root is None:
 return 0

 left_depth = max_depth(root.left)
 right_depth = max_depth(root.right)

 return 1 + max(left_depth, right_depth)
```

```

Complexity

Every node is visited once:

```
```text
Time = O(n)
```
```

Recursion stack:

```
```text
Space = O(h)
```
```

Q23. Could Maximum Depth be solved using BFS?

Answer

Yes.

BFS can count the number of levels.

For example:

```
```text
Level 1 → 3
Level 2 → 9, 20
Level 3 → 15, 7
```
```

Therefore:

```
```text
number of levels = maximum depth
```
```

This is a useful interview point:

A problem may have multiple valid patterns. The goal is to choose and explain a suitable one.

Q24. What are common mistakes in tree problems?

Answer

Common mistakes include:

1. Forgetting the base case

```
```python
if node is None:
 return ...
```
```

2. Mixing traversal orders

For example, implementing:

```
```text
Left → Root → Right
```
```

when the problem requires:

```
```text
Root → Left → Right
```
```

3. Incorrect queue-level handling

Processing newly added children as though they belong to the current level.

4. Incorrect complexity

Saying:

```
```text
DFS = $O(\log n)$
```
```

is wrong in general.

Traversal still visits every node:

```
```text
 $O(n)$
```
```

5. Confusing height and depth

Always establish the definition being used.

PART V — BACKEND QUERY ABSTRACTION

Verbatim conversation content — formatted for textbook presentation

PART V — BACKEND QUERY ABSTRACTION

Day 9 moves beyond the basic CRUD/query implementation from Day 8.

The central architectural idea is:

```
```text
HTTP Query Parameters
 ↓
Validated Query Representation
 ↓
Service
 ↓
Repository
 ↓
Parameterized SQL
 ↓
PostgreSQL
```
```

Q25. Why should SQL not be written directly in the FastAPI router?

Answer

The router's primary responsibility is handling HTTP concerns.

For example:

```
```text
HTTP request
validation
query parameters
HTTP response
status codes
```
```

SQL is a persistence concern.

If SQL is placed directly in the router, several problems arise:

- HTTP and database logic become coupled.
- Testing becomes harder.
- Business logic becomes difficult to reuse.
- SQL logic gets duplicated.
- The router becomes increasingly large.
- Changing the persistence layer becomes harder.

A cleaner separation is:

```
```text
Router
 ↓
Service
 ↓
Repository
 ↓
Database
```
```

This separation gives each layer a clear responsibility.

Q26. What responsibilities belong to the router?

Answer

The router should primarily handle:

- HTTP request
- Path parameters
- Query parameters
- Request validation
- Calling the service layer
- Mapping results to HTTP responses
- HTTP status codes

Conceptually:

```
```text
HTTP
↓
Router
```
```

The router should not become the location where database query construction is implemented.

Q27. What responsibilities belong to the service layer?

Answer

The service layer handles:

- Business rules
- Business validation
- Orchestration
- Calling repositories
- Coordinating operations

For example:

```
```text
Router
↓
DocumentService
↓
DocumentRepository
```
```

The service should not become a second SQL layer.

Q28. What responsibilities belong to the repository?

Answer

The repository is responsible for persistence.

It can handle:

- SQL construction

- Database execution
- Parameter binding
- Mapping database rows
- Persistence-specific behavior

Therefore:

```
```text
Repository → PostgreSQL
```
```

This separation gives each layer a clear responsibility.

Q29. What problem occurs if we create a repository method for every filter combination?

Imagine we initially have:

```
```python
get_all()
```
```

Then we add:

```
```python
get_by_type()
get_by_status()
get_sorted()
get_paginated()
get_by_type_sorted()
get_by_type_paginated()
get_by_status_sorted()
get_by_status_paginated()
get_by_type_status_sorted()
...
```
```

The number of combinations can grow rapidly.

This is method explosion.

The repository becomes difficult to maintain.

Q30. What is a query object?

Answer

Instead of representing each query combination as a different method, represent the query itself as structured data.

For example:

```
```python
class DocumentQuery:
 document_type: str | None
 status: str | None
 limit: int
 offset: int
 sort_field: str
 sort_order: str
...
```
```

Then the flow becomes:

```
```text
HTTP parameters
 ↓
DocumentQuery
 ↓
DocumentService
 ↓
DocumentRepository
 ↓
SQL
```
```

This lets one repository method interpret different combinations of query requirements.

Q31. Why is a query object better than many repository methods?

Answer

It separates:

```
```text
"What does the caller want?"
```
```

from:

```
```text
"How do we execute that query?"
```
```

For example:

```
```text
DocumentQuery
 filter = specification
 pagination = specification
 ordering = specification
```
```

The repository can then translate the structured query into SQL.

This reduces:

- Duplication
 - Method explosion
 - Coupling
 - Maintenance cost
-

Q32. Does the query object mean the router should construct SQL?

Answer

No.

The router can create or receive a validated query representation, but it should not construct SQL.

The architecture remains:

```
```text
Router
↓
validated query representation
↓
Service
↓
Repository
↓
SQL
```
```

The query object represents intent, not SQL.

Q33. How do you safely implement dynamic sorting?

This is an important security question.

Suppose the user sends:

```
```text
?sort=name
```
```

Values can be parameterized:

```
```sql
WHERE document_type = %s
```
```

But SQL identifiers such as:

```
```sql
ORDER BY name
```
```

cannot simply be handled as normal parameter values.

Therefore use an allowlist.

Example:

```
```python
ALLOWED_SORT_FIELDS = {
 "name": "name",
 "created_at": "created_at",
 "document_type": "document_type",
}
```
```

Then:

```
```text
user input
↓
validation
↓
allowlist
↓
safe SQL identifier
```
```

Unknown fields should be rejected.

Q34. Why can't we simply parameterize the column name?

Answer

Parameterized queries generally bind values, not arbitrary SQL syntax or identifiers.

For example:

```
```sql
WHERE document_type = %s
```
```

works for values.

But conceptually:

```
```sql
ORDER BY %s
```
```

does not mean "replace this with a column identifier" in the same way.

Therefore dynamic identifiers should be selected from trusted application-defined mappings.

Q35. What is SQL injection?

Answer

SQL injection occurs when untrusted input is incorrectly incorporated into SQL syntax.

For example, unsafe construction could conceptually look like:

```
```python
query = f"SELECT * FROM documents WHERE name = '{user_input}'"
```
```

If the input contains SQL syntax, it can alter the intended query.

The safer approach is:

```
```python
query = """
SELECT *
FROM documents
WHERE name = %s
"""

params = (user_input,)
```
```

The database driver handles the value separately from the SQL structure.

Q36. Is parameterization alone enough for every dynamic SQL component?

Answer

No.

We need to distinguish:

Values

Use parameterization:

```
```sql
WHERE name = %s
```
```

SQL identifiers

Use an allowlist:

```
```python
ALLOWED_SORT_FIELDS = {
 "name": "name",
 "created_at": "created_at",
}
```
```

This distinction is important for secure dynamic SQL.

Q37. Walk me through GET /documents.

Strong interview answer

I would explain the request flow end-to-end:

```
```text
HTTP Request
 ↓
FastAPI Router
 ↓
Pydantic validation
 ↓
DocumentQuery
 ↓
DocumentService
 ↓
DocumentRepository
 ↓
Validated query construction
 ↓
Parameterized SQL
 ↓
PostgreSQL
 ↓
Rows
 ↓
Domain/API representation
 ↓
HTTP Response
```
```

The router handles HTTP concerns.

The service handles business logic and orchestration.

The query object represents the requested filtering, pagination and ordering.

The repository converts that structured query into safe SQL and executes it.

Values are parameterized and dynamic SQL identifiers such as sort fields are selected through an allowlist.

Finally, the database result is returned through the service and router.

Q38. What happens if the query has an invalid sort field?

Answer

The invalid input should be rejected at the application boundary.

For example:

```
```text
sort=unknown_column
```
```

should not become:

```
```sql
ORDER BY unknown_column
```
```

Instead:

```
```text
Request
 ↓
Validation
 ↓
Unknown sort field
 ↓
Client error
```
```

The important principle is:

Invalid input should be rejected before it reaches unsafe database query construction.

PART VI — API RELIABILITY

Verbatim conversation content — formatted for textbook presentation

PART VI — API RELIABILITY

Q39. What does API reliability mean?

Answer

Reliability means that the API behaves predictably when things go wrong, not just when everything works.

Examples of failures include:

- Database unavailable
- Database timeout
- Invalid input
- Unexpected application exception
- Network failure
- Client retry
- Partial operation failure

A reliable API should:

```
```text
detect failure
 ↓
handle failure
 ↓
return an appropriate response
 ↓
avoid corrupting state
 ↓
provide useful observability
```
```

Q40. What happens if PostgreSQL becomes unavailable?

Strong answer

Suppose:

```
```text
GET /documents
```
```

is received.

The flow becomes:

```
```text
Request
 ↓
Router
 ↓
Service
 ↓
Repository
 ↓
PostgreSQL connection
 ↓
Failure
```
```

The application should not return:

```
```json
{
 "status": "success"
}
```
```

when the database operation actually failed.

Instead, the failure should propagate through the application's error-handling mechanism and result in an appropriate API error response.

The exact HTTP response depends on the application's error-handling policy, but the key principle is:

Never report successful persistence or retrieval when the underlying operation failed.

Q41. Why should database failures be tested?

Answer

Most applications are tested primarily on successful paths:

```
```text
request → success
```
```

But production failures often occur at boundaries.

For example:

```
```text
API
↓
Database
```
```

The database can fail independently of the application.

Therefore reliability tests should verify:

```
```text
database unavailable
 ↓
repository failure
 ↓
service behavior
 ↓
API error behavior
```
```

This ensures failure behavior is deliberate rather than accidental.

Q42. What is a timeout?

Answer

A timeout places an upper bound on how long an operation is allowed to wait.

Without appropriate timeout behavior:

```
```text
API request
 ↓
database operation
 ↓
database hangs
 ↓
request waits indefinitely
```
```

This can consume:

- Threads
- Workers
- Connections
- Memory
- Other system resources

A timeout converts an indefinitely waiting operation into a controlled failure.

Q43. Why are timeouts important in distributed systems?

Answer

Because dependencies can fail in ways that are not immediately visible.

For example:

```
```text
API
 ↓
Service
 ↓
Database
```
```

The database may be:

- Slow
- Unreachable
- Overloaded
- Waiting on another resource

Without bounded waiting, the API can accumulate stuck requests.

Therefore timeout design is part of reliability.

Q44. What is idempotency?

Answer

An operation is idempotent if performing it multiple times has the same intended final effect as performing it once.

For example, conceptually:

```
```text
DELETE document 123
```
```

If the first request succeeds and the client does not receive the response because of a network failure, the client may retry.

The second DELETE should not create an additional destructive effect.

This makes retry behavior easier to reason about.

Q45. Why are retries dangerous?

Answer

Suppose a client sends:

```
```text
POST /payment
```
```

The server processes the request, but the response is lost.

The client cannot distinguish:

```
```text
request failed
```
```

from:

```
```text
request succeeded but response was lost
```
```

If the client blindly retries, the operation may happen twice.

Therefore retries need to be considered together with:

- Idempotency
- Request identity
- State transitions
- Timeouts
- Partial failures

Q46. Is DELETE always idempotent?

Answer

HTTP semantics generally treat DELETE as idempotent in the sense that repeating the request should result in the same intended resource state.

But implementation details still matter.

For example:

```
```text
DELETE /documents/123
```
```

First call:

```
```text
document exists → deleted
```
```

Second call:

```
```text
document already absent
```
```

The system should define what response semantics it wants for the second request.

The important distinction is between idempotent effect and identical response for every invocation.

Q47. What reliability tests should you write for Day 9?

Answer

At minimum:

Test 1 — Database failure

Verify that a PostgreSQL failure does not produce a false success.

Test 2 — Unexpected repository exception

Verify that unexpected failures follow the application's global error behavior.

Test 3 — Invalid query parameters

Verify invalid values are rejected before unsafe database operations.

Test 4 — Query abstraction regression

Verify the new query representation produces the same expected results as the previous implementation.

Test 5 — Ordering validation

Verify unsupported sort fields are rejected.

PART VII — SYSTEM DESIGN

Verbatim conversation content — formatted for textbook presentation

PART VII — SYSTEM DESIGN

Q48. Design a Document Hierarchy Service.

Answer

For Day 9, start with:

```
```text
Project
|
+-- Requirement
|
+-- Design
|
+-- Code Change
|
+-- Test
```
```

The goal is to model relationships among engineering artifacts.

Q49. What requirements would you identify first?

Answer

Functional requirements:

- Create a node
- Get a node
- List children
- Retrieve descendants
- Associate engineering artifacts
- Maintain relationships between artifacts

For example:

```
```text
Create node
Get node
List children
Get descendants
```
```

Non-functional requirements include:

- Reliability
- Query performance
- Consistency
- Observability
- Scalability
- Access control

Q50. What APIs might you design?

Answer

Conceptually:

```
```text
POST /nodes
GET /nodes/{id}
GET /nodes/{id}/children
GET /nodes/{id}/descendants
```
```

The exact API design depends on requirements.

The important interview skill is explaining why the API boundaries exist.

Q51. How would you represent the hierarchy?

Answer

A simple relational representation could use:

```
```text
id
parent_id
type
name
metadata
created_at
```
```

For example:

```
```text
Requirement
|
+-- Design
|
+-- Code Change
```
```

Each child can reference its parent:

```
```text
parent_id
```
```

This can represent a tree-like hierarchy without immediately introducing a graph database.

Q52. Would you use a graph database immediately?

Answer

Not necessarily.

The current Day 9 objective is to understand:

```
```text
hierarchy
relationships
traversal
data modeling
```
```

A relational database can represent many hierarchical structures.

The technology choice should follow requirements such as:

- Relationship complexity
- Traversal depth
- Query patterns
- Scale
- Consistency requirements
- Operational constraints

The project should not introduce a graph database simply because the data has relationships.

Q53. How does BFS/DFS relate to the document hierarchy?

Answer

Suppose:

```
```text
Project
 ↓
Requirement
 ↓
Design
 ↓
Code Change
 ↓
Test
```
```

If we want to retrieve descendants recursively, DFS is natural:

```
```text
Project
 ↓
Requirement
 ↓
Design
 ↓
Code Change
 ↓
Test
```
```

If we want nodes organized by hierarchy level:

```
```text
Level 0 → Project
Level 1 → Requirements
Level 2 → Designs
Level 3 → Code Changes
Level 4 → Tests
```
```

BFS becomes natural.

Therefore the DSA concepts directly support system design reasoning.

Q54. How does this relate to Project 1?

Answer

Project 1 ultimately wants traceability:

```
```text
Requirement
 ↓
Code
 ↓
Test
 ↓
Result
```
```

Day 9 introduces the underlying engineering concept:

```
```text
hierarchical / relationship-aware data
```
```

The goal is not to implement the entire traceability graph now.

Instead, understand the data relationships so later features such as:

```
```text
impact analysis
test generation
change analysis
traceability
```
```

have a strong architectural foundation.

PART VIII — PROJECT INTERVIEW QUESTIONS

Verbatim conversation content — formatted for textbook presentation

PART VIII — PROJECT INTERVIEW QUESTIONS

Q55. Explain the architecture of your current backend.

Strong answer

The backend follows a layered architecture:

```
```text
FastAPI Router
 ↓
Service Layer
 ↓
Repository Layer
 ↓
PostgreSQL
```
```

The router handles HTTP concerns and request validation.

The service layer handles business logic and orchestration.

The repository owns persistence and SQL.

For document retrieval, query parameters are converted into a structured query representation, which is passed through the service into the repository.

The repository safely constructs parameterized SQL and executes it against PostgreSQL.

This separation prevents HTTP concerns from becoming coupled to persistence logic.

Q56. Why did you choose layered architecture?

Answer

The primary reason is separation of responsibilities.

Without separation, a single endpoint could contain:

```
```text
HTTP validation
business logic
SQL construction
database execution
error handling
```
```

That becomes difficult to test and maintain.

With layers:

```
```text
Router
```
```

handles HTTP.

```
```text
Service
```
```

handles business behavior.

```
```text
Repository
```
```

handles persistence.

Each layer has a narrower responsibility.

Q57. Why introduce DocumentQuery?

Strong answer

Initially, filtering, pagination and sorting can be implemented through repository methods.

As combinations increase, creating a repository method for every combination creates method explosion.

For example:

```
```text
get_all
get_filtered
get_sorted
get_paginated
get_filtered_sorted
get_filtered_paginated
```
```

Instead, I represent query intent as structured data:

```
```text
DocumentQuery
```
```

The repository then interprets that query and builds the appropriate SQL.

This allows query complexity to grow without creating a separate repository method for every combination.

Q58. How do you prevent SQL injection in your implementation?

Answer

I separate SQL structure from user-controlled values.

Values are passed through parameterized queries:

```
```sql
WHERE document_type = %s
```
```

For dynamic SQL identifiers such as sorting fields, I use an explicit allowlist:

```
```python
ALLOWED_SORT_FIELDS = {
 "name": "name",
 "created_at": "created_at",
}
```
```

Therefore user input cannot directly become arbitrary SQL syntax.

Q59. What happens if PostgreSQL is unavailable?

Strong answer

The repository encounters a database failure.

That failure propagates through the service to the API's error-handling mechanism.

The API must not return a successful response when the database operation failed.

I would also have a test that simulates database failure and verifies that the expected error behavior occurs.

Q60. How would you explain your backend to an interviewer in 60 seconds?

Model answer

"The backend is a FastAPI application using a layered architecture of Router, Service and Repository, with PostgreSQL as the persistence layer. The router owns HTTP concerns and validation, the service owns business logic and orchestration, and the repository owns SQL and database execution. For document retrieval, I use a structured query representation so filtering, pagination and ordering don't cause repository method explosion. Values are parameterized, and dynamic sort fields are restricted through an allowlist. I also test failure paths such as database failures and invalid query parameters rather than testing only successful requests."

This is a strong Day 9 project explanation because it combines:

```
```text
Architecture
+
Security
+
Maintainability
+
Testing
+
Reliability
```
```

PART IX — GOOGLE-STYLE FOLLOW-UP QUESTIONS

Verbatim conversation content — formatted for textbook presentation

PART IX — GOOGLE-STYLE FOLLOW-UP QUESTIONS

These are the questions where an interviewer may continue probing after your initial answer.

Q61. Why not put all business logic in the repository?

Answer

Because persistence logic and business logic have different responsibilities.

The repository should answer:

How do I store or retrieve this data?

The service should answer:

What should the application do?

Keeping them separate makes business rules easier to test independently of PostgreSQL.

Q62. Why not skip the service layer?

Answer

For a very small application, a service layer may appear unnecessary.

But as the system grows, business orchestration tends to increase.

The service layer provides a boundary between:

```
```text
HTTP
```
```

and:

```
```text
Persistence
```
```

It also allows business logic to be reused by other interfaces later.

Q63. What if you later add Redis?

Answer

Redis would be introduced for a specific requirement such as:

- Caching
- Rate limiting
- Short-lived state
- Queue-related coordination

It should not be introduced merely because it is part of the eventual technology stack.

The current Day 9 objective is to make the PostgreSQL-backed service boundary clean first.

Q64. What if the database becomes slow?

Answer

I would investigate systematically:

```
```text
Application latency
 ↓
Database latency
 ↓
Query execution
 ↓
Indexes
 ↓
Query plan
 ↓
Connection pool
 ↓
Database resource utilization
```
```

I would measure before optimizing.

Potential actions could include:

- Query optimization
- Appropriate indexes
- Pagination
- Connection pool tuning
- Caching where justified
- Query result reduction

The exact solution depends on measured bottlenecks.

Q65. What if your repository method becomes too large?

Answer

That is a sign that query construction responsibilities may need further decomposition.

For example:

```
```text
DocumentRepository
 ↓
query construction
filter construction
ordering construction
pagination construction
execution
mapping
```
```

These responsibilities can potentially be separated internally while keeping the public repository boundary clean.

The goal is not to create abstractions for their own sake.

Q66. What is the difference between an abstraction and unnecessary complexity?

Answer

An abstraction is useful when it manages a real source of complexity.

For example:

```
```text
multiple combinations of filters
+
sorting
+
pagination
```
```

can justify a query representation.

But adding:

```
```text
Factory
Builder
Manager
Strategy
Adapter
Registry
```
```

without a real problem creates unnecessary complexity.

The rule for Day 9 is:

Introduce abstraction where it reduces actual complexity in the current system.

PART X — RAPID-FIRE QUESTIONS

Verbatim conversation content — formatted for textbook presentation

PART X — RAPID-FIRE QUESTIONS

You should be able to answer these in 15–30 seconds each.

Q67. What is DFS?

Answer: Depth-first traversal that explores one branch deeply before moving to another, typically implemented with recursion or a stack.

Q68. What is BFS?

Answer: Breadth-first traversal that processes nodes level by level, typically implemented using a queue.

Q69. DFS data structure?

Answer: Stack, either explicitly or implicitly through recursion.

Q70. BFS data structure?

Answer: Queue.

Q71. Preorder?

Answer:

```
```text
Root → Left → Right
```
```

Q72. Inorder?

Answer:

```
```text
Left → Root → Right
```
```

Q73. Postorder?

Answer:

```
```text
Left → Right → Root
```
```

Q74. Tree traversal time complexity?

Answer:

```
```text
O(n)
```
```

because every node is visited.

Q75. Recursive DFS auxiliary space?

Answer:

```
```text
O(h)
```
```

where h is tree height.

Q76. Why use a queue for BFS?

Answer: FIFO ordering naturally processes earlier-level nodes before their children.

Q77. What indicates BFS?

Answer: Level-by-level or shortest-path-in-unweighted-graph reasoning.

Q78. What indicates DFS?

Answer: Subtree, recursion, deep exploration or combining child results.

Q79. What is query abstraction?

Answer: Representing query intent as structured data rather than creating a repository method for every combination of filters, pagination and sorting.

Q80. Why parameterized SQL?

Answer: To separate SQL structure from user-controlled values and reduce SQL injection risk.

Q81. How do you safely implement dynamic ORDER BY?

Answer: Validate the requested sort field against an application-controlled allowlist.

Q82. Router responsibility?

Answer: HTTP handling and request/response validation.

Q83. Service responsibility?

Answer: Business logic and orchestration.

Q84. Repository responsibility?

Answer: Persistence, SQL construction/execution and database mapping.

Q85. What is idempotency?

Answer: Repeating an operation produces the same intended final state as performing it once.

Q86. Why are timeouts important?

Answer: They prevent requests from waiting indefinitely on slow or unavailable dependencies.

PART XI — SCENARIO-BASED INTERVIEW QUESTIONS

Verbatim conversation content — formatted for textbook presentation

PART XI — SCENARIO-BASED INTERVIEW QUESTIONS

These are particularly important for Google-style interviews.

Q87. The interviewer says: "Your API is working. Why do you care about failure cases?"

Answer

A production system doesn't operate only under ideal conditions.

Dependencies can fail:

```
```text
Database
Network
External service
Application process
```
```

Therefore correctness includes both:

```
```text
success behavior
+
failure behavior
```
```

For example, if PostgreSQL is unavailable, returning a successful response would be incorrect even though the API process itself is still running.

Q88. Your API receives 1,000 requests with invalid sort fields. What should happen?

Answer

The application should validate the sort field before attempting database execution.

For example:

```
```text
sort=invalid
```
```

should fail validation.

The application should not construct:

```
```sql
ORDER BY invalid
```
```

from arbitrary user input.

This protects the query boundary and avoids unnecessary database work.

Q89. An interviewer asks: "Why not just use an ORM?"

Answer

The important issue isn't whether raw SQL or an ORM is universally better.

The architectural principle is:

```
```text
HTTP concerns
≠
business logic
≠
persistence logic
```
```

An ORM can still live behind the repository boundary.

For this project, understanding parameterized SQL and query construction is valuable because it demonstrates understanding of what the persistence layer is actually doing.

Q90. The interviewer asks: "What happens if a client retries a failed DELETE?"

Answer

I would first distinguish whether the original request:

```
```text
actually failed
```
```

or:

```
```text
succeeded but the response was lost
```
```

A retry can therefore encounter an already-deleted resource.

The operation should be designed with idempotent semantics so repeating the operation does not cause additional destructive effects.

The API's exact response semantics should also be explicitly defined.

PART XII — DEEP PROJECT QUESTION

Verbatim conversation content — formatted for textbook presentation

PART XII — DEEP PROJECT QUESTION

Q91. Why is Day 9 relevant to an AI Engineering platform?

Answer

Because the AI portion of the system eventually depends on strong software engineering foundations.

Project 1 will eventually include:

```
```text
Requirements
Code
Tests
Results
Retrieval
Agents
Evaluation
Observability
```
```

Those components require:

- Clean APIs
- Database modeling
- Query abstraction
- Error handling
- Reliability
- Testing
- Data traversal

For example:

```
```text
Requirement
↓
Design
↓
Code Change
↓
Test
↓
Result
```
```

is a hierarchical/relationship-oriented structure.

Therefore the tree, DFS and BFS concepts from Day 9 are not isolated DSA exercises—they develop reasoning that will later support the architecture of Project 1.

PART XIII — MOST IMPORTANT INTERVIEW CONNECTIONS

Verbatim conversation content — formatted for textbook presentation

PART XIII — MOST IMPORTANT INTERVIEW CONNECTIONS

You should be able to explain these connections without notes.

Connection 1

```
```text
Tree
↓
DFS
↓
Recursion
↓
Subtree computation
```
```

Example:

```
```text
Maximum Depth
```
```

Connection 2

```
```text
Tree
↓
BFS
↓
Queue
↓
Level processing
```
```

Example:

```
```text
Level Order Traversal
```
```

Connection 3

```
```text
HTTP Query Parameters
↓
Validation
↓
DocumentQuery
↓
Service
↓
Repository
↓
Parameterized SQL
↓
PostgreSQL
```
```

Connection 4

```
```text
Database Failure
↓
Repository Error
↓
Service
↓
Error Handling
↓
API Response
```
```

Connection 5

```
```text
Dynamic Sorting
↓
Validation
↓
Allowlist
↓
Safe SQL
```
```

Connection 6

```
```text
Project Hierarchy
↓
Requirements
↓
Design
↓
Code
↓
Tests
↓
Results
```
```

This eventually connects to Project 1's:

```
```text
Requirement → Code → Test → Result
```
```

traceability objective.

PART XIV — DAY 9 MOCK INTERVIEW

Verbatim conversation content — formatted for textbook presentation

PART XIV — DAY 9 MOCK INTERVIEW

Do this without looking at the answers.

DSA Round

1. What is a tree?
2. What is a binary tree?
3. Explain depth versus height.
4. Explain DFS.
5. Why does DFS naturally use recursion?
6. Explain BFS.
7. Why does BFS use a queue?
8. Explain preorder, inorder and postorder.
9. When would you choose BFS over DFS?
10. Implement preorder traversal.
11. Implement level-order traversal.
12. Implement maximum depth.
13. Give the time and space complexity of each.
14. Explain the invariant of your level-order solution.
15. Could maximum depth be solved using BFS?

Backend Round

16. Explain Router → Service → Repository.
17. Why shouldn't SQL be in the router?
18. What is query abstraction?
19. Why does repository method explosion occur?
20. What is DocumentQuery?
21. How do you safely implement dynamic sorting?
22. What is SQL injection?
23. Why use parameterized queries?
24. How do you validate query parameters?
25. Walk through GET /documents.

Reliability Round

26. What happens when PostgreSQL is unavailable?
27. Why are database failure tests necessary?
28. What is a timeout?
29. Why do distributed systems need timeouts?
30. What is idempotency?
31. Why are retries dangerous?

- 32. What happens if DELETE is retried?
- 33. What reliability tests would you add?

System Design Round

- 34. Design a document hierarchy service.
 - 35. What APIs would you expose?
 - 36. How would you represent the hierarchy?
 - 37. How would you retrieve descendants?
 - 38. When would DFS be useful?
 - 39. When would BFS be useful?
 - 40. Would you immediately introduce a graph database?
 - 41. How does this architecture support Project 1?
 - 42. How would you monitor this service?
-

PART XV — THE 10 QUESTIONS YOU MUST MASTER

Verbatim conversation content — formatted for textbook presentation

PART XV — THE 10 QUESTIONS YOU MUST MASTER

If your Day 9 interview preparation time is limited, master these first:

1. Explain DFS and BFS.

You should immediately state:

```
```text
DFS → stack/recursion → deep exploration
BFS → queue → level exploration
```
```

2. Explain preorder/inorder/postorder.

```
```text
Preorder = Root Left Right
Inorder = Left Root Right
Postorder = Left Right Root
```
```

3. Solve level-order traversal.

You must understand:

```
```python
level_size = len(queue)
```
```

and why it matters.

4. Solve maximum depth.

Know:

```
```text
depth(node) = 1 + max(left, right)
```
```

5. Explain tree traversal complexity.

```
```text
Time = O(n)
Space = O(h) for recursive DFS
```
```

or appropriate queue-width space for BFS.

6. Explain Router → Service → Repository.

Be able to defend every boundary.

7. Explain query abstraction.

The key idea:

```
```text
Query intent as data
```
```

instead of:

```
```text
One repository method per query combination
```
```

8. Explain safe dynamic SQL.

```
```text
Values → parameterization
Identifiers → allowlist
```
```

9. Explain PostgreSQL failure behavior.

Never return false success when persistence failed.

10. Walk through GET /documents.

Memorize the architecture, not a script:

```
```text
HTTP Request
 ↓
Router
 ↓
Validation
 ↓
DocumentQuery
 ↓
Service
 ↓
Repository
 ↓
Parameterized SQL
 ↓
PostgreSQL
 ↓
Result
 ↓
HTTP Response
```
```

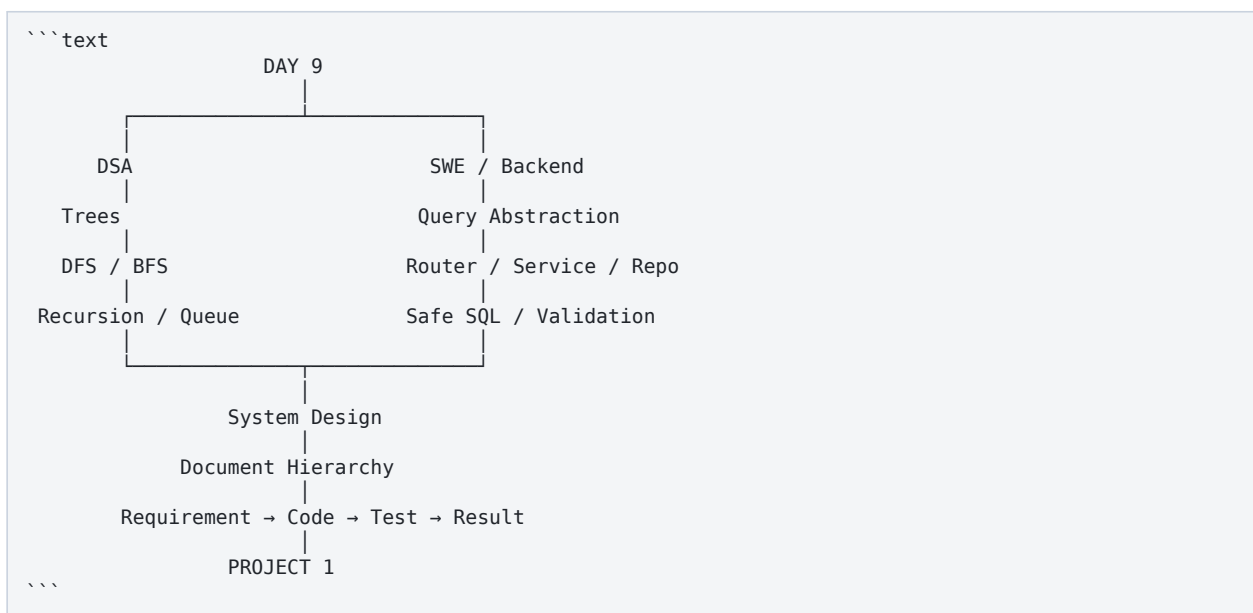
Day 9 Interview Definition of Done

Before considering the interview portion of Day 9 complete, you should be able to do all of the following without notes:

- Explain tree terminology.
- Draw and explain a binary tree.
- Explain DFS and BFS.
- Explain why DFS uses recursion/stack.
- Explain why BFS uses a queue.
- Explain preorder/inorder/postorder.

- Recognize DFS versus BFS from problem wording.
- Solve preorder traversal.
- Solve level-order traversal.
- Solve maximum depth.
- Explain $O(n)$ traversal complexity.
- Explain $O(h)$ recursive DFS space.
- Explain Router → Service → Repository.
- Explain why SQL belongs in the repository.
- Explain repository method explosion.
- Explain DocumentQuery.
- Explain parameterized SQL.
- Explain sort-field allowlisting.
- Explain SQL injection.
- Explain database failure handling.
- Explain timeout reasoning.
- Explain idempotency.
- Explain retry risks.
- Design a basic document hierarchy.
- Connect tree traversal to Project 1.
- Walk through GET /documents from HTTP request to PostgreSQL.
- Defend your architectural decisions when challenged.

The most important Day 9 interview skill is being able to connect the DSA and engineering concepts rather than treating them as separate study topics:



That is the level of understanding to target for Day 9—not merely being able to recite definitions.

VERBATIM CONVERSATION — END

Supplementary Learning Aids

The following material is generated as a study aid. It is not part of the verbatim conversation and does not replace or modify any source content above.

Chapter Summaries

Part I — Trees, DFS & BFS

Build the core mental model for trees, binary trees, depth versus height, DFS, BFS, and traversal recognition. The key interview skill is choosing DFS for subtree/recursive reasoning and BFS for level-based reasoning.

Part II — Binary Tree Preorder

Know preorder as Root → Left → Right, understand the recursive call stack, and be able to derive the iterative stack solution.

Part III — Binary Tree Level Order

Use a queue and freeze the current level size before processing it. This level boundary is the central implementation detail.

Part IV — Maximum Depth

Understand the recurrence $\text{depth}(\text{node}) = 1 + \max(\text{left}, \text{right})$, the base case, and $O(n)$ traversal with $O(h)$ recursive space.

Part V — Backend Query Abstraction

Keep HTTP, business logic, and persistence concerns separated. Represent query intent as structured data and prevent repository method explosion.

Part VI — API Reliability

Reason about database failures, timeouts, retries, idempotency, and tests that validate failure behavior rather than only success paths.

Part VII — System Design

Model a document hierarchy using explicit requirements, APIs, relational data, traversal strategy, and operational constraints.

Part VIII — Project Interview Questions

Defend the current FastAPI architecture, explain DocumentQuery, secure dynamic SQL, and describe database failure behavior.

Part IX — Google-Style Follow-Ups

Expect probing questions about service boundaries, Redis justification, database slowness, repository size, and abstraction trade-offs.

Part X — Rapid-Fire Questions

Practice concise 15–30 second answers for the core definitions, responsibilities, complexity, security, and reliability concepts.

Part XI — Scenario-Based Questions

Demonstrate production reasoning: invalid inputs, dependency failures, retries, ORM trade-offs, and idempotent DELETE semantics.

Part XII — Deep Project Question

Connect the Day 9 DSA and backend work directly to the AI Engineering platform and its eventual traceability architecture.

Part XIII — Most Important Connections

Unify DSA and SWE: tree→DFS, tree→BFS, HTTP→query object→repository, database failure→error handling, and hierarchy→traceability.

Part XIV — Day 9 Mock Interview

Use the 42-question mock interview as an active recall drill across DSA, backend, reliability, and system design.

Part XV — The 10 Questions You Must Master

Prioritize the ten highest-value explanations: DFS/BFS, traversals, level order, maximum depth, complexity, architecture, query abstraction, safe SQL, DB failure, and GET /documents.

Visual Study Summaries

1. DSA Pattern Selection

| Problem signal | Likely pattern | Core data structure |
|--|----------------|---------------------|
| Subtree / recursive child computation | DFS | Recursion / Stack |
| Level-by-level / nearest in unweighted traversal | BFS | Queue |
| Root before children | Preorder | DFS |
| Children before parent | Postorder | DFS |

2. Project 1 Request Flow

```
graph TD
    A[HTTP Request] --> B[FastAPI Router — HTTP concerns + validation]
    B --> C[DocumentQuery — structured query intent]
    C --> D[Service — business logic + orchestration]
    D --> E[Repository — safe SQL + persistence]
    E --> F[PostgreSQL]
    F --> G[Result → HTTP Response]
```

3. Safe Dynamic SQL

| User input | → | Validation / Allowlist | → | Safe SQL |
|-------------|---|--------------------------------|---|-------------------------|
| Values | → | Parameterized binding | → | WHERE ... = %s |
| Identifiers | → | Application-controlled mapping | → | ORDER BY trusted_column |

Revision Checklist

- ☐ Explain tree, binary tree, depth, and height without notes.
- ☐ Explain DFS and BFS and justify the data structure behind each.
- ☐ Write preorder traversal recursively and iteratively.
- ☐ Write level-order traversal and explain why `len(queue)` defines a level.
- ☐ Derive maximum depth recursively and state its complexity.
- ☐ Explain Router → Service → Repository and defend each boundary.
- ☐ Explain DocumentQuery and repository method explosion.
- ☐ Explain parameterized SQL versus allowlisted SQL identifiers.
- ☐ Explain SQL injection and how the query boundary prevents it.
- ☐ Explain PostgreSQL failure, timeout, retry, and idempotency behavior.
- ☐ Design a basic document hierarchy service.
- ☐ Walk through GET /documents end-to-end.
- ☐ Answer the 42-question mock interview aloud.
- ☐ Deliver the 60-second backend explanation without reading it.

Glossary

| Term | Study definition |
|---------------------|---|
| Binary Tree | A tree in which each node has at most two children. |
| BFS | Breadth-First Search; traversal that processes nodes level by level using a queue. |
| DFS | Depth-First Search; traversal that explores deeply before moving to another branch. |
| Depth | Number of edges from the root to a node. |
| Height | Number of edges on the longest downward path from a node to a leaf, subject to the convention used. |
| Idempotency | The property that repeating an operation has the same intended final effect as performing it once. |
| Query Object | Structured data representing query intent such as filters, pagination, and ordering. |
| Repository | Persistence boundary responsible for database access and persistence-specific operations. |
| Service Layer | Application boundary responsible for business logic and orchestration. |
| Parameterized Query | A query where values are bound separately from SQL structure. |
| Allowlist | An explicit set of application-approved values or identifiers. |
| Timeout | A bound on how long an operation is allowed to wait. |
| Method Explosion | Rapid growth in specialized methods caused by combinations of filters, sorting, and pagination. |

Index

- API reliability
- BFS
- binary tree
- database failure
- depth
- DFS
- DocumentQuery
- idempotency
- level-order traversal
- maximum depth
- parameterized SQL
- PostgreSQL
- preorder traversal
- query abstraction
- repository
- retries
- Router → Service → Repository
- SQL injection
- system design
- timeouts
- tree
- Project 1

Index note: page-level index locations are intentionally omitted from this generated aid because the source conversation was preserved as a formatted transcript rather than rewritten into a new reference text.